U.PORTO
FEUP FACULDADE DE ENGENHARIA
UNIVERSIDADE DO PORTO

Ferramenta de Alocação de Registos em Compiladores

Projeto de Programação II
Desenho de Algoritmos (DA)

Francisco Lourenço - up202404899
José Lopes - up202406360
Rodrigo Roque - up202403623

01
Introdução

O Problema Prático

Contexto: Num compilador, após gerar o código intermédio em 3-address instructions, é necessário mapear as variáveis do programa para os registos físicos do processador.

Objetivo: Decidir quais variáveis ficam em que registos e quais vão para memória.

Restrições: Duas variáveis não podem partilhar o mesmo registo se estiverem vivas simultaneamente. O problema de encontrar o mínimo número de registos necessários é equivalente ao problema de coloração de grafos — onde cada cor corresponde a um registo.

O Problema Prático

Abordagem: Desenvolvemos uma ferramenta que implementa quatro estratégias de alocação:

- Basic
- Spilling
- Splitting
- Free

02
Estruturas de Dados

Estruturas de Dados

Para além da estrutura de grafo fornecida, definimos as seguintes estruturas de dados de suporte:

- VariableRanges e LivePoints
- Web
- RegisterConfig
- ColoringResult

VariableRanges e LivePoints

- **LiveRangePoint** especifica o que acontece numa dada linha (se é definição **isDef**, ou se é a morte da variável **isEnd**).
- **VariableRanges** agrupa todos os pontos e os intervalos da variável de acordo com o ficheiro de texto de entrada.

```
struct LiveRangePoint {
    int line;
    bool isDef; //se é a linha onde foi definido
    bool isEnd; //se é a linha onde morreu
};

struct VariableRanges {
    std::string varName;
    std::vector<std::vector<LiveRangePoint>> ranges;
};
```

Web

- **id:** Identificador único do Web.
- **originalVariable:** Nome da variável original.
- **activeLines, defLines, useLines:** Vetores de inteiros que indicam, respetivamente, em que linhas de código o Web está ativo, onde é definido e onde é utilizado (onde morre e variável).

```
struct Web {
    int id;
    std::string originalVariable;
    std::vector<int> activeLines;
    std::vector<int> defLines;
    std::vector<int> useLines;
};
```

RegisterConfig

Estrutura simples para armazenar a configuração lida do ficheiro de registos.

- **numRegisters:** Número total de registos.
- **algorithmName:** Algoritmo a ser usado.
- **algorithmParam:** Parâmetro extra como o limite de spills/splits.

```
struct RegisterConfig {
    int numRegisters = 0;
    std::string algorithmName;
    int algorithmParam = 0;
};
```

ColoringResult

Estrutura devolvida por todos os algoritmos de alocação. Armazena o estado final da tentativa de alocação:

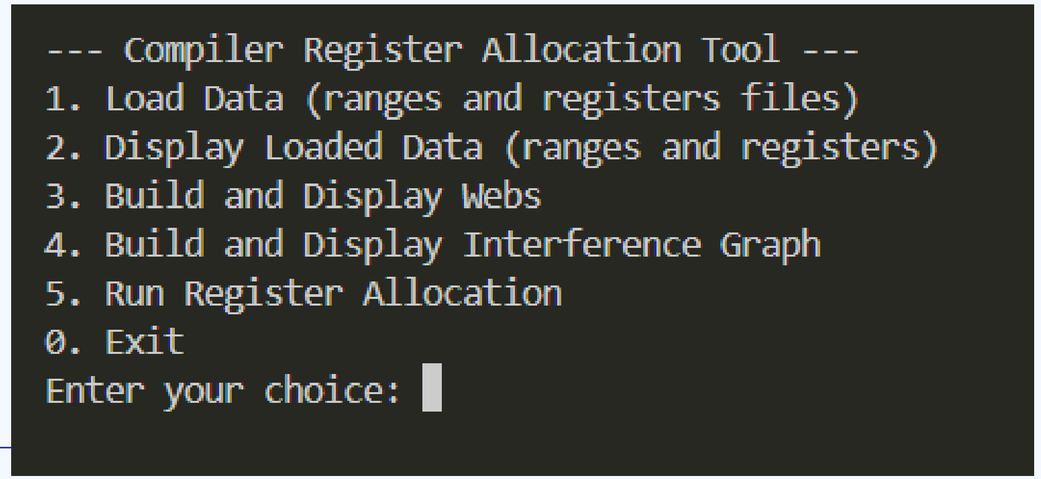
- **success:** Booleano que indica se a coloração funcionou sem spills forçados.
- **numColorsUsed:** Quantidade de registos (cores) efetivamente utilizados.
- **webToRegister:** Um unordered_map<int, int> que mapeia o ID do Web para o Registo atribuído (ou -1 se foi para a memória).
- **spilledWebs:** Lista de IDs de webs que tiveram de ir para memória (Spill).
- **splitWebs:** Lista com as novas instâncias de Webs criadas quando a estratégia é de Splitting (partição do tempo de vida).

```
struct ColoringResult {
    bool success;                                   //
    int numColorsUsed;                              //
    std::unordered_map<int, int> webToRegister;     //
    std::vector<int> spilledWebs;                   //
    std::vector<Web> splitWebs;                     //
};
```

03
Menu

Menu

5 opções organizadas de forma sequencial — cada uma depende da anterior:
O estado é preservado entre opções — webs e grafo são construídos uma vez e reutilizados. A opção 1 pede também o ficheiro de output, que é usado na opção 5 para escrever o resultado.
A opção 5 delega em AllocationEngine::runAllocation que seleciona automaticamente o algoritmo com base no algorithmName do ficheiro de registos — basic, spilling, splitting ou free.



04
Pipeline do Programa

Pipeline do Programa

Input

Processa (ranges e registers).txt

Webs

Fuzão das live ranges em webs

Grafo de Interferência

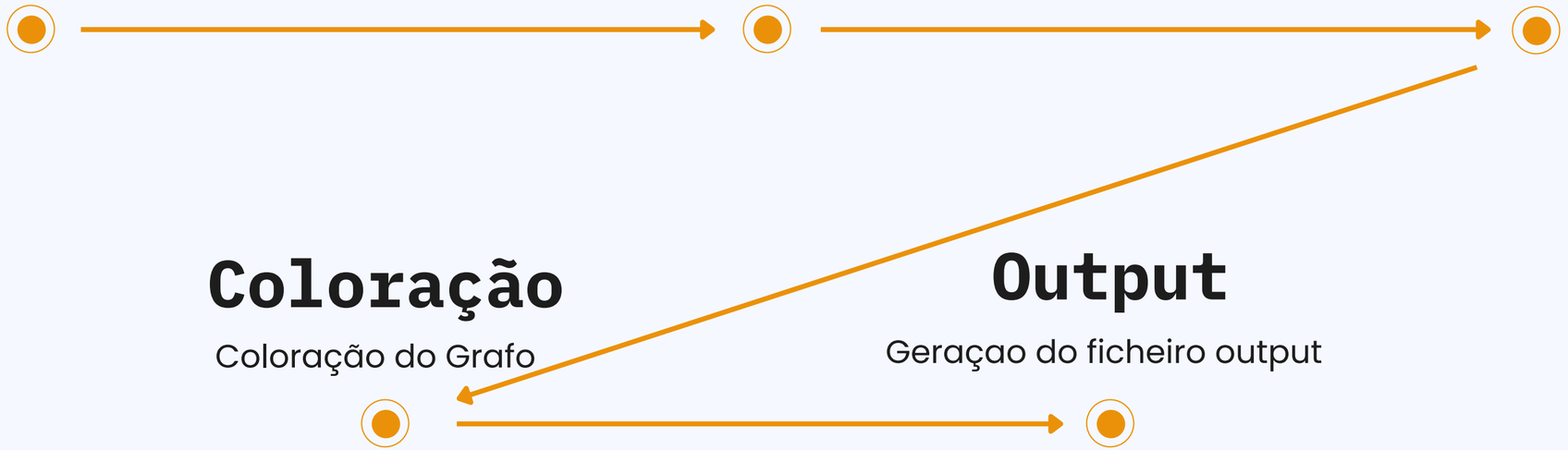
Construçao do Grafo de Interferência

Coloração

Coloração do Grafo

Output

Geraçao do ficheiro output



Processamento dos Dados

O DataLoader é composto por 3 funções principais:

- **resolvePath:** usada para retornar o path correto para os respetivos ficheiros de input;
- **loadRanges:** usada para carregar as informações do ficheiros dos ranges; e
- **loadRegisters:** usada para carregar as informações do ficheiros dos registers.

Tanto o loadRanges como o loadRegisters lêm os ficheiros linha a linha ignorando linhas vazias e comentários.

```
56    while (std::getline(file, line)) {
57        line = trim(line);
58        // Ignore empty lines and comments
59        if (line.empty() || line[0] == '#') {
60            continue;
61        }
```

Processamento dos Dados

No **loadRanges** para cada linha separa pelo “**:**” para obter o nome da variável e a lista de pontos. Depois separa a lista pelo “**,**” e para cada token verifica o último caractere.

```
169             if (token.back() == '+') {
170               point.isDef = true;
171               token.pop_back();
172             } else if (token.back() == '-') {
173               point.isEnd = true;
174               token.pop_back();
175             }
```

O número restante é convertido com **std::stoi** e adicionado ao **VariableRanges** da variável correspondente. Cada linha do ficheiro gera um novo **vector<LiveRangePoint>** que é adicionado ao ranges da variável.

Processamento dos Dados

Separa pelo “:” e processa duas chaves:

- **registers** → converte diretamente para config.numRegisters com std::stoi.
- **algorithm** → verifica se existe vírgula na parte do valor:

```
80          } else if (key == "algorithm") {
81            size_t commaPos = valueStr.find(',');
82
83            if (commaPos != std::string::npos) {
84              config.algorithmName = trim(valueStr.substr(0, commaPos));
85              try {
86                config.algorithmParam =
87                    std::stoi(trim(valueStr.substr(commaPos + 1)));
88              } catch (const std::exception &e) {
89                std::cerr << "Error: Bad format for algorithm parameter in "
90                          << filename << std::endl;
91                throw std::runtime_error("Bad format in " + filename);
92              }
93            } else {
94              config.algorithmName = valueStr;
95
96              if (config.algorithmName == "spilling" ||
97                  config.algorithmName == "splitting") {
98
99                throw std::runtime_error(config.algorithmName +
100                                         " requires a parameter K (e.g. " +
101                                         config.algorithmName + ", 2)");
102             }
103           }
```

Complexidade

$$O(N \times L)$$

- **N** = Número de linhas.
- **L** = Número máximo de caracteres numa linha

Criação das Webs

Nesta fase criamos uma struct temporária para ajudar na sobreposiçao de live ranges.

```
11      struct TempWeb {
12          std::set<int> activeLines;
13          std::set<int> defLines;
14          std::set<int> useLines;
15      };
```

Para cada variável, itera pelos seus live ranges. Para cada range cria um TempWeb e verifica sobreposição com os já existentes:

```
for (int line : currentRange.activeLines) {
    if (existing.activeLines.count(line)) {
        overlap = true;
        break;
    }
}
```

Criação das Webs

Se há sobreposição, absorve o **existing** no **combined**. Se não, mantém-no separado.
No fim adiciona o combined ao **nextMergedRanges.**

```
if (overlap) {
    // Combine into the current range
    combined.activeLines.insert(existing.activeLines.begin(), existing.activeLines.end());
    combined.defLines.insert(existing.defLines.begin(), existing.defLines.end());
    combined.useLines.insert(existing.useLines.begin(), existing.useLines.end());
} else {
    // No overlap, keep it separate
    nextMergedRanges.push_back(existing);
}
```

No fim converte todas as Webs temporárias para Webs normais.

```
// Convert the final merged ranges into Web structs
for (const auto& tw : mergedRanges) {
    Web web;
    web.id = nextWebId++;
    web.originalVariable = varName;

    web.activeLines.assign(tw.activeLines.begin(), tw.activeLines.end());
    web.defLines.assign(tw.defLines.begin(), tw.defLines.end());
    web.useLines.assign(tw.useLines.begin(), tw.useLines.end());

    // Vectors from std::set are already sorted, but we ensure sorting anyway
    std::sort(web.activeLines.begin(), web.activeLines.end());
    std::sort(web.defLines.begin(), web.defLines.end());
    std::sort(web.useLines.begin(), web.useLines.end());

    allWebs.push_back(web);
}
```

Complexidade

$$O(V \times R^2 \times L \times \log L)$$

- **V** = Número de variáveis.
- **R** = Número de ranges por variável.
- **L** = Número de linhas por range.

Criação do Grafo de Interferência

Para cada par de webs (a, b) usa **std::set_intersection** para encontrar as linhas partilhadas — possível em **O(L)** porque as activeLines estão ordenadas:

```
std::vector<int> sharedLines;
std::set_intersection(a.activeLines.begin(), a.activeLines.end(),
                      b.activeLines.begin(), b.activeLines.end(),
                      std::back_inserter(sharedLines));
```

Dois webs não interferem numa linha L se um termina (-) e o outro começa (+) nessa mesma linha. Para cada linha partilhada L verificamos se se aplica essa exceção:

```
bool allExceptions = true;
for (int L : sharedLines) {
    bool aUsesBDefs = std::binary_search(a.useLines.begin(), a.useLines.end(), L) &&
                      std::binary_search(b.defLines.begin(), b.defLines.end(), L);

    bool bUsesADefs = std::binary_search(b.useLines.begin(), b.useLines.end(), L) &&
                      std::binary_search(a.defLines.begin(), a.defLines.end(), L);

    if (!aUsesBDefs && !bUsesADefs) {
        allExceptions = false;
        break;
    }
}
```

Criação do Grafo de Interferência

Depois a criação do grafo é trivial.

```
// Add one vertex per web (using web ID as info)
for (const auto& web : webs) {
    graph.addVertex(web.id);
}

// Calculate interferences and add edges
auto interferences = calculateInterferences(webs);
for (const auto& [id1, id2] : interferences) {
    graph.addBidirectionalEdge(id1, id2, 1.0);
}
```

Complexidade

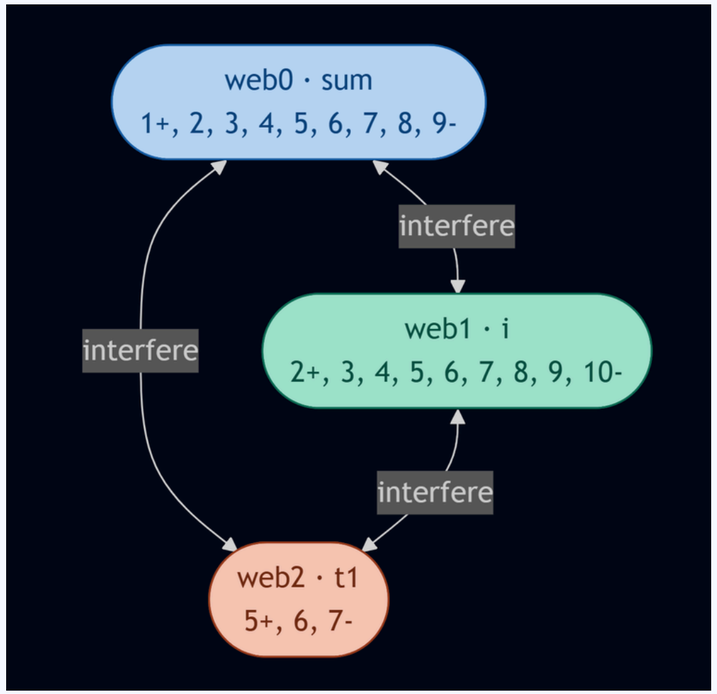
$$O(W^2 \times L)$$

- **W** = Número de webs.
- **L** = Número de linhas por web.

Exemplo

sum: 1+,2,3,4,5,6,7,8,9-
i: 2+,3,4,5,6,7,8-
i: 8+,9,10-
t1: 5+,6,7-

web0 (sum): 1+,2,3,4,5,6,7,8,9-
web1 (i): 2+,3,4,5,6,7,8,9,10-
web2 (t1): 5+,6,7-



05
Algoritmos

Basic

Calculamos os graus iniciais e identificamos logo quais os nós triviais (grau < N). Usa um vertexMap para lookups O(1):

```cpp
// Dynamic degrees and trivial nodes initialization: O(V + E)
std::unordered_map<int, int> currentDegrees;
std::vector<int> trivialNodes;

for (int id : active) {
    int deg = 0;
    auto it = vertexMap.find(id);
    if (it != vertexMap.end() && it->second != nullptr) {
        for (auto* edge : it->second->getAdj()) {
            if (active.count(edge->getDest()->getInfo())) {
                deg++;
            }
        }
    }
    currentDegrees[id] = deg;

    if (deg < numRegisters) {
        trivialNodes.push_back(id);
    }
}
```

Basic

Se há nós triviais, remove um e atualiza os graus dos vizinhos. Se algum vizinho passar a ter grau N-1, torna-se trivial imediatamente — sem scanning O(V):

```
// Simplification Loop: O(V + E)
while (!active.empty()) {
    if (!trivialNodes.empty()) {
        int id = trivialNodes.back();
        trivialNodes.pop_back();

        if (!active.count(id)) {
            continue;
        }

        colorStack.push(id);
        active.erase(id);

        // Decrement degrees of active neighbors
        auto it = vertexMap.find(id);
        if (it != vertexMap.end() && it->second != nullptr) {
            for (auto* edge : it->second->getAdj()) {
                int neighborId = edge->getDest()->getInfo();
                if (active.count(neighborId)) {
                    currentDegrees[neighborId]--;
                    if (currentDegrees[neighborId] == numRegisters - 1) {
                        trivialNodes.push_back(neighborId);
                    }
                }
            }
        }
```

Se não há nós triviais, todos têm grau ≥ N — faz spill do maior grau.

Basic

Na fase de coloração dá pop da stack e para cada nó recolhe as cores já atribuídas aos seus vizinhos. (Quando um node é spilled não o removemmos do grafo nem criamos um novo - apenas colocamos a sua color = -1).

```
std::vector<int> neighborColors;
auto itLookup = vertexMap.find(id);
if (itLookup != vertexMap.end() && itLookup->second != nullptr) {
    for (auto* edge : itLookup->second->getAdj()) {
        int neighborId = edge->getDest()->getInfo();
        auto it = colorAssignment.find(neighborId);
        if (it != colorAssignment.end() && it->second >= 0) {
            neighborColors.push_back(it->second);
        }
    }
}
```

Depois chama assignColor que itera de 0 até N-1 e devolve a primeira cor não usada pelos vizinhos. Se vier -1 é porque não mais cores para distribuir.

Spilling

O algoritmo é estruturalmente igual ao básico — mesma inicialização, mesma fase de simplificação, mesma fase de coloração. As duas diferenças são:

1. Limite K — quando não há nós triviais, verifica primeiro se já atingiu o máximo de spills permitidos:

```
// Prevent excessive spilling.
if ((int)forcedSpills.size() >= maxSpills) {
  std::cerr << "WARNING: Spill limit of " << maxSpills
            << " exceeded. Allocation not possible.\n";
  result.success = false;
  return result;
}
```

2. Critério de seleção inteligente — em vez do maior grau, usa a heurística de Chaitin

Heurística de Chaitin

métrica = custo / grau

- **Custo** = número de definições + usos do web
- **Grau** = número de interferências ativas

```
double cost = calculateSpillCost(*it->second);
double metric = cost / degree;
```

Prefere webs com baixo custo (pouco usados em runtime) e alto grau (removem muitas interferências). Grau 0 é tratado como 1 para evitar divisão por zero.

Splitting

Escolhe o web com maior grau no grafo atual. Em caso de empate, prefere o web com mais activeLines — mais linhas = mais opções de corte:

```
// Prefer higher degree; break ties by preferring longer webs
if (deg > bestDegree || (deg == bestDegree && size > bestSize)) {
    bestDegree = deg;
    bestId = id;
    bestSize = size;
}
```

Para cada posição de corte possível, conta quantos outros webs estão ativos nessa linha. Escolhe o corte com menor interferência — minimiza os webs que ficam dos dois lados da fronteira. Em caso de empate, prefere o corte mais próximo do meio:

```
// Prefer lower interference; break ties by preferring the
// cut closest to the midpoint (most balanced split)
int curDist = cutIdx > midpoint ? cutIdx - midpoint : midpoint - cutIdx;
int bestDist = bestCutIdx > midpoint ? bestCutIdx - midpoint : midpoint - bestCutIdx;
if (interference < minInterference ||
    (interference == minInterference && curDist < bestDist)) {
    minInterference = interference;
    bestCutIdx = cutIdx;
}
```

Splitting

De seguida cria duas sub webs a partir da antiga e ao invés de criar um grafo novo todas as vezes, remove apenas o node antigo e adiciona os dois novos.

```
currentGraph.removeVertex(candidateId);
currentGraph.addVertex(webA.id);
currentGraph.addVertex(webB.id);
```

Se após K splits ainda falha, todos os webs (incluindo os sub-webs criados) vão para memória.

Free

Para o algoritmo livre optámos por implementar uma variante de optimistic coloring com Kempe Chain rescue. Em vez de fazer spill imediato quando um nó não tem cor disponível, o algoritmo tenta primeiro reordenar as cores já atribuídas aos vizinhos — libertando uma cor para o nó bloqueado sem violar nenhuma restrição de interferência. Só desiste e faz spill se o rescue for impossível.

Free

A fase de simplificação é quase igual ao básico — mas quando não há nós triviais, em vez de fazer spill, faz optimistic coloring: empurra o nó para a stack na mesma, apostando que a coloração vai funcionar:

```
else {
  // Optimistic coloring: push highest-degree node onto stack anyway
  int bestCandidate = -1;
  int bestDegree = -1;
  for (int id : active) {
      int deg = currentDegrees[id];
      if (deg > bestDegree) {
          bestDegree = deg;
          bestCandidate = id;
      }
  }

  if (bestCandidate == -1) {
      result.success = false;
      break;
  }

  colorStack.push(bestCandidate);
  active.erase(bestCandidate);
```

Free

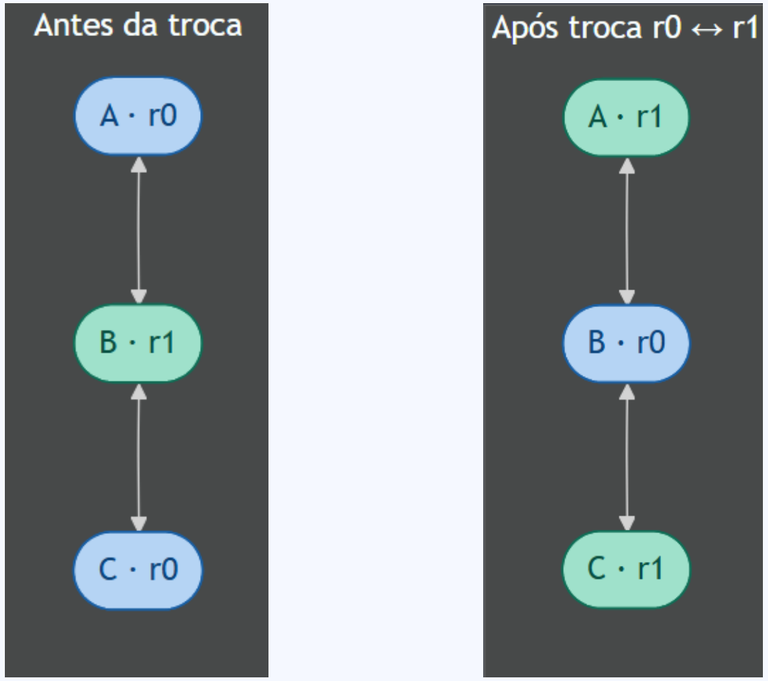
Tenta atribuir cor normalmente. Se todas as cores estão ocupadas pelos vizinhos, em vez de spill imediato tenta resgatar o nó com Kempe Chain:

```
if (color == -1) {
    // All colors taken — attempt Kempe Chain rescue
    std::cout << "  [Kempe] Node " << id
              << " stuck (all " << numRegisters
              << " registers taken). Attempting rescue...\n";

    color = tryKempeRescue(graph, id, colorAssignment, numRegisters);
```

Kempe Chain

Uma Kempe Chain é um conjunto de nós já coloridos, ligados entre si no grafo, que usam apenas duas cores c1 e c2. A propriedade fundamental é que trocar c1 ↔ c2 em todos os nós da chain é sempre válido — nós dentro da chain só têm vizinhos com c1 ou c2, por isso a troca não cria conflitos.



Free

De forma resumida, **tryKempeRescue** é chamado na fase de coloração sempre que assignColor devolve -1.

Internamente, para cada par de cores **(c, c2)** usadas pelos vizinhos, chama **buildKempeChain** que faz um BFS construindo o conjunto de nós conectados que usam apenas essas duas cores.

Se a chain resultante não contiver nenhum vizinho com cor **c2**, a troca **c ↔ c2** é executada em toda a chain, libertando c para o nó bloqueado. Se nenhum par de cores produzir uma chain válida, o nó vai para memória.

Complexidade

Algoritmo	**Fase 1**	**Fase 2**	**Total**
Basic	$O(V + E)$	$O(V + E)$	$\mathbf{O(V + E)}$
Spilling	$O(V + E)$	$O(V + E)$	$\mathbf{O(V + E)}$
Splitting	$O(K \times W \times L)$	$O(V + E)$	$\mathbf{O(K \times W \times L)}$
Free	$O(V + E)$	$O(V \times N^2 \times (V + E))$	$\mathbf{O(V \times N^2 \times (V + E))}$

- **V =** número de webs
- **E** = número de interferências
- **K** = número máximo de splits
- **W** = número de webs no grafo atual (após split)
- **L** = número de linhas por web
- **N** = número de registos disponíveis

Output

Gera o ficheiro de output no formato especificado pelo enunciado. Dois casos:

- Alocação bem sucedida — agrupa os webs por registo, iterando de r0 até ao último registo usado. Webs spilled durante o processo (color = -1) aparecem no fim com M.
- Alocação falhada — registers: 0 e todos os webs com M.

```
if (!result.success) {
  outFile << "registers: 0\n";
  for (const auto &web : webs) {
    outFile << "M: web" << web.id << "\n";
  }
```